

GenAI Interview Q&A For BHRMS Chatbot

This guide is based on this project: FastAPI, WebSocket streaming, Celery, Redis, PostgreSQL/pgvector, local HuggingFace embeddings, CrossEncoder reranking, Ollama LLM, and multi-tenant RAG.

Architecture Questions

1. Explain the end-to-end flow of this GenAI chatbot.

Answer: Tenant admins upload documents, Celery processes and embeds them into pgvector, users connect through WebSocket, the backend retrieves relevant chunks, reranks them, sends context to the LLM, streams the answer, and saves conversation and usage logs.

2. What happens when a user uploads a document?

Answer: The API validates the PDF, stores it, calculates a hash, checks duplicates per tenant, creates a DB row with PROCESSING status, and queues a Celery task to extract, chunk, embed, and store vectors.

3. What happens when a user sends a chat message through WebSocket?

Answer: The system validates agent access, builds or reuses an agent service, parses the message, retrieves relevant knowledge if it is a RAG agent, streams LLM tokens, then stores messages and usage data.

4. Why are embeddings, reranker, LLM manager, and vector DB initialized as shared resources?

Answer: They are expensive to load. Sharing them reduces startup cost per request and avoids repeatedly loading large ML models.

5. What is the role of FastAPI lifespan in this project?

Answer: Lifespan initializes Redis clients, shared AI resources, semantic cache, and cleans them up during shutdown.

6. Why is Celery used instead of processing documents inside the API request?

Answer: PDF extraction and embedding are slow CPU/GPU-heavy tasks. Celery keeps upload requests responsive and processes documents asynchronously.

7. How does the system support multi-tenancy?

Answer: Tenant ID is stored in users, documents, conversations, messages, usage logs, vector metadata, and Redis cache prefixes.

8. How do you isolate documents between tenants?

Answer: Every vector chunk includes tenant_id metadata, and retrieval filters by tenant_id before generating answers.

9. What is the difference between a chat/RAG agent and a generative-only agent?

Answer: A RAG agent retrieves context from documents before answering. A generative-only agent answers from the model prompt without document retrieval.

10. Why is Redis used in this architecture?

Answer: Redis is used for Celery broker/backend, token blacklist checks, chat history, and semantic caching.

RAG Questions

1. What is RAG?

Answer: Retrieval-Augmented Generation combines information retrieval with LLM generation, so answers are grounded in external documents.

2. Why do we need RAG instead of directly asking the LLM?

Answer: LLMs may not know private HR policies and may hallucinate. RAG supplies the latest tenant-specific context.

3. Explain the RAG pipeline used in this project.

Answer: PDF text is extracted, chunked, embedded, stored in pgvector, retrieved by query similarity, reranked, inserted into the prompt, and answered by the LLM.

4. What are the main stages: document ingestion, chunking, embedding, retrieval, reranking, generation?

Answer: Ingestion loads documents, chunking splits text, embedding converts chunks to vectors, retrieval finds candidates, reranking improves relevance, and generation produces the final answer.

5. What is the purpose of chunking?

Answer: Chunking makes long documents searchable and small enough to fit into model context.

6. How do you decide chunk size and overlap?

Answer: Choose chunk size based on document structure, retrieval quality, and LLM context size. Overlap preserves continuity between adjacent chunks.

7. What happens if chunks are too small?

Answer: They may lose context, causing incomplete or misleading answers.

8. What happens if chunks are too large?

Answer: Retrieval becomes less precise, context window is wasted, and irrelevant text may confuse the LLM.

9. How do you attach metadata to chunks?

Answer: During processing, each node gets metadata such as tenant_id, category, source file, page label, file hash, doc_id, and chunk index.

10. Why is metadata filtering important in RAG?

Answer: It enforces tenant isolation, filters by category/source, and improves retrieval precision.

Vector Database Questions

1. Why did you use pgvector?

Answer: pgvector keeps vector search inside PostgreSQL, allowing relational data and vector search in one system.

2. What is a vector database?

Answer: A vector database stores embeddings and supports similarity search over high-dimensional vectors.

3. What is cosine similarity?

Answer: Cosine similarity measures the angle between vectors to estimate semantic closeness.

4. What is HNSW indexing?

Answer: HNSW is an approximate nearest-neighbor index that speeds up vector search with good recall.

5. What is hybrid search?

Answer: Hybrid search combines semantic vector search with keyword/sparse search for better retrieval.

6. How do you filter vectors by tenant id?

Answer: Store tenant_id in vector metadata and apply a MetadataFilter during retrieval.

7. How do you delete all vectors related to a document?

Answer: Delete vector nodes using filters for tenant_id and file_hash or doc_id.

8. What is embedding dimension, and why must it match the model?

Answer: It is the vector length produced by the embedding model. pgvector table dimension must match or inserts/queries fail.

9. What production issues can happen if embedding dimension is wrong?

Answer: Indexing fails, retrieval breaks, or existing vectors become incompatible after model changes.

10. How would you scale vector search for many tenants?

Answer: Use indexes, metadata filters, table partitioning if needed, connection pooling, query limits, and separate vector stores for very large tenants.

Embedding Questions

1. What is an embedding?

Answer: An embedding is a numeric vector representation of text that captures semantic meaning.

2. Which embedding model is used in this project?

Answer: The model comes from settings.EMBEDDING_MODEL and is loaded through HuggingFaceEmbedding.

3. Why use local HuggingFace embeddings?

Answer: They support offline/private deployment and reduce external API cost.

4. What is the difference between embedding model and LLM?

Answer: The embedding model converts text to vectors for search. The LLM generates natural language answers.

5. Why should the same embedding model be used for indexing and querying?

Answer: Query vectors and document vectors must live in the same semantic space.

6. What happens if you change embedding model after documents are indexed?

Answer: Existing vectors become inconsistent. You should re-embed all documents.

7. What is batch embedding?

Answer: Batch embedding processes multiple texts together for better throughput.

8. Why use GPU if available?

Answer: GPU speeds up embedding and reranking, especially for large documents and high traffic.

9. How do you handle offline model loading?

Answer: Store model artifacts locally and set HuggingFace/Transformers offline mode.

10. What are pros and cons of local embeddings vs OpenAI embeddings?

Answer: Local embeddings improve privacy and cost control but need infrastructure. API embeddings are easier to operate but cost money and send data externally.

Reranking Questions

1. What is reranking?

Answer: Reranking reorders retrieved candidate chunks using a stronger relevance model.

2. Why do we need a reranker after vector search?

Answer: Vector search is fast but approximate. A reranker improves final context quality.

3. What is a CrossEncoder?

Answer: A CrossEncoder scores a query and document together, making relevance judgment more accurate.

4. Difference between bi-encoder and cross-encoder?

Answer: Bi-encoders create separate vectors for fast search. Cross-encoders compare query-document pairs directly and are slower but more accurate.

5. Why is reranking slower but more accurate?

Answer: It evaluates each query-document pair jointly instead of using precomputed vectors only.

6. How many documents should you rerank?

Answer: Usually rerank a small candidate set, such as top 10 or 20, then keep top 3 to 5.

7. What happens if the reranker model is unavailable?

Answer: The system can fall back to vector similarity ranking, but answer quality may drop.

8. How can reranking improve answer quality?

Answer: It selects more relevant chunks, reducing irrelevant context and hallucination risk.

9. What is the tradeoff between retrieval top-k and reranking top-k?

Answer: Higher top-k improves recall but increases latency. Lower top-k is faster but may miss useful context.

10. Where is reranking used in this code?

Answer: In ChatRAGAgentService, after initial retrieval and before creating LangChain documents for answer generation.

LLM Questions

1. Which LLM is used in this project?

Answer: The project uses Ollama through LangChain ChatOllama, with the model configured by settings.LLM_MODEL.

2. Why use Ollama?

Answer: Ollama allows local LLM serving, useful for privacy, cost control, and offline deployments.

3. What is temperature?

Answer: Temperature controls randomness. Lower values make answers more deterministic; higher values make answers more creative.

4. What is num_predict or max token limit?

Answer: It limits the maximum number of generated output tokens.

5. What is streaming response?

Answer: Streaming sends tokens as they are generated instead of waiting for the full answer.

6. Why stream tokens over WebSocket?

Answer: WebSocket supports real-time bidirectional communication and improves perceived response speed.

7. How do you handle model unavailability?

Answer: Return a clear error, avoid crashing the connection, and monitor model health.

8. What is the difference between local LLM and cloud LLM?

Answer: Local LLMs give privacy and control but need hardware. Cloud LLMs are easier to scale but add cost and external dependency.

9. What are stop tokens?

Answer: Stop tokens tell the model where to stop generation to avoid unwanted continuations.

10. How would you reduce hallucinations?

Answer: Use strict prompts, good retrieval, reranking, metadata filters, citations, low temperature, and answer-only-from-context rules.

Prompt Engineering Questions

1. What is a system prompt?

Answer: It is a high-priority instruction that defines the assistant role, behavior, constraints, and tone.

2. How do you pass tenant-specific or agent-specific instructions?

Answer: Agent configuration stores system prompt, tone, model, and allowed sources, which are inserted into the prompt chain.

3. How do you enforce tone?

Answer: Add tone instructions to the system prompt and keep them consistent across agent responses.

4. How do you make the model answer only from context?

Answer: Prompt it to use only provided context and say it does not know when context is insufficient.

5. What is query rewriting?

Answer: Query rewriting transforms a conversational question into a standalone search query.

6. Why do we rewrite a user query before retrieval?

Answer: Follow-up questions may depend on chat history. Rewriting improves retrieval accuracy.

7. What is a history-aware retriever?

Answer: It uses chat history to reformulate the current query before searching documents.

8. How do you prevent prompt injection from documents?

Answer: Treat document text as untrusted data, use system instructions that ignore document commands, and avoid executing content from documents.

9. How do you handle "I don't know" responses?

Answer: If retrieved context is weak or missing, instruct the model to say it cannot answer from available documents.

10. How do you improve factual grounding?

Answer: Improve chunking, retrieval, reranking, citations, prompt constraints, and evaluation metrics.

Document Processing Questions

1. How is a PDF processed in this project?

Answer: It is converted to Markdown/text, page markers are added, content is split into nodes, metadata is attached, and nodes are embedded into pgvector.

2. Why use Docling?

Answer: Docling preserves layout and tables better than basic PDF text extraction.

3. Why is there a fallback to pypdf?

Answer: If Docling fails, pypdf provides a simpler extraction path so processing can still continue.

4. How are page numbers preserved?

Answer: Page markers like PAGE_START are inserted before splitting, then converted into page_label metadata.

5. How are tables handled?

Answer: The processor exports PDF pages to Markdown and applies table cleanup before chunking.

6. Why add source and page metadata into chunk text?

Answer: It helps the LLM cite or explain where the answer came from and improves grounding.

7. How do you prevent duplicate document uploads?

Answer: Calculate a file hash and check whether the same tenant already has a document with that hash.

8. Why calculate file hash?

Answer: It identifies duplicate content and can be used to delete vectors belonging to a file.

9. What happens if document processing fails?

Answer: The Celery task retries. If processing ultimately fails, the document status should be marked FAILED.

10. How would you process Word, Excel, or scanned PDFs?

Answer: Add file-type-specific parsers and OCR for scanned PDFs, then normalize extracted content into the same chunking and embedding pipeline.

Celery Questions

1. Why use Celery for document processing?

Answer: It moves slow background work out of the API request cycle.

2. What is a Celery broker?

Answer: A broker is a message queue that receives tasks from producers and delivers them to workers.

3. Why is Redis used as broker/backend?

Answer: Redis is simple, fast, and already used by the app for cache and chat history.

4. What happens when a Celery task fails?

Answer: The task raises an exception and Celery retries according to max_retries and countdown settings.

5. What does max_retries=3 do?

Answer: It limits the task to three retry attempts before final failure.

6. Why is import-time model loading risky in Celery?

Answer: It loads heavy models whenever the module is imported, including in API processes, wasting memory and slowing startup.

7. How would you monitor Celery tasks?

Answer: Use Flower, logs, task status fields in DB, metrics, and alerting on failure/retry counts.

8. How do you update document status after processing?

Answer: The worker opens a DB session, finds the document row, and updates processing_status.

9. What is the issue with running too many Celery workers with local models?

Answer: Each worker may load its own models, causing high RAM/GPU usage and possible crashes.

10. How would you avoid duplicate processing jobs?

Answer: Use file hash checks, DB uniqueness constraints, idempotent tasks, and possibly distributed locks.

Redis Questions

1. Where is Redis used in this project?

Answer: Redis is used for Celery, semantic cache, chat history, and JWT blacklist checks.

2. What is semantic caching?

Answer: It caches LLM responses by semantic similarity rather than exact string match.

3. What is chat history TTL?

Answer: It is the expiry time for stored chat messages in Redis.

4. Why store chat history in Redis?

Answer: Redis is fast and suitable for temporary session memory.

5. How does token blacklist work?

Answer: Logged-out or revoked tokens are stored in Redis, and each request checks whether the token is blacklisted.

6. Difference between sync and async Redis clients?

Answer: Async clients are used in async FastAPI flows. Sync clients are useful for libraries that do not support async.

7. What happens if Redis goes down?

Answer: Celery, blacklist checks, chat history, and semantic cache may fail or degrade.

8. How would you make Redis production ready?

Answer: Use authentication, TLS, persistence, managed Redis or clustering, monitoring, memory limits, and eviction policy.

9. What is cache invalidation in RAG?

Answer: It means clearing stale cached answers when documents, prompts, or permissions change.

10. How do you isolate cache per tenant?

Answer: Use tenant-specific Redis key prefixes.

FastAPI / Backend Questions

1. Why use FastAPI?

Answer: It is async-friendly, fast, supports dependency injection, Pydantic validation, OpenAPI docs, and WebSockets.

2. What is dependency injection in FastAPI?

Answer: It provides dependencies like DB sessions, current user, and shared services automatically to route handlers.

3. What is app.state used for?

Answer: It stores application-wide resources like Redis clients and shared AI models.

4. What is FastAPI lifespan?

Answer: It is startup/shutdown lifecycle management for initializing and cleaning resources.

5. Difference between HTTP endpoint and WebSocket endpoint?

Answer: HTTP is request-response. WebSocket keeps a persistent connection for real-time streaming.

6. Why is WebSocket suitable for streaming GenAI responses?

Answer: It lets the server send tokens as they are generated and allows interactive chat sessions.

7. How is authentication handled?

Answer: HTTP endpoints use JWT bearer tokens. WebSocket validates token query parameters for private agents.

8. What is role-based access control?

Answer: RBAC restricts actions based on user roles, such as tenant_admin.

9. How does require_role("tenant_admin") work?

Answer: It gets the current user and checks whether the user's role name is allowed.

10. How do you handle DB sessions safely?

Answer: Use dependency-managed sessions, commit on success, rollback on errors, and close sessions after use.

Production Questions

1. What are the main production risks in this codebase?

Answer: Open CORS, secret exposure, duplicate model loading, import-time model loading, WebSocket close bugs, Docker config mismatch, and missing robust failure handling.

2. Why is allow_origins=["*"] risky?

Answer: It allows any website to call the API, which is unsafe especially with authenticated requests.

3. Why should secrets not be inside Docker Compose?

Answer: Compose files are often committed or shared. Secrets should come from environment or a secret manager.

4. Why can multiple Uvicorn workers be dangerous with local AI models?

Answer: Each worker loads its own models, multiplying memory and GPU usage.

5. How would you monitor latency, errors, and token usage?

Answer: Use metrics middleware, Prometheus/Grafana, structured logs, DB usage logs, and alerts.

6. How would you handle model warmup?

Answer: Load models during startup and optionally run a small test inference before marking the service ready.

7. How would you deploy this system?

Answer: Use separate services for API, Celery worker, Redis, Postgres/pgvector, Ollama/model server, and monitoring.

8. How would you scale chat traffic?

Answer: Scale API workers carefully, externalize model serving, use connection limits, rate limits, and horizontal replicas.

9. How would you scale document ingestion?

Answer: Scale Celery workers based on queue size, limit concurrency for model memory, and separate CPU/OCR workloads from embedding workloads.

10. How would you handle large PDFs?

Answer: Enforce upload limits, process page by page, stream files, use background jobs, and store processing progress.

Security Questions

1. How is JWT used here?

Answer: JWT identifies users and tenants, and is checked before protected API access or private WebSocket chat.

2. How do you prevent one tenant from accessing another tenant's documents?

Answer: Validate tenant access in auth and filter vector retrieval by tenant_id.

3. What is the risk of public agents?

Answer: Public agents can be abused, may expose unintended information, and require rate limiting and strict retrieval filters.

4. Why should WebSocket token validation be strict?

Answer: WebSocket connections are long-lived and can stream sensitive data.

5. What is prompt injection?

Answer: Prompt injection is when user or document content tries to override system instructions.

6. How can uploaded documents become an attack vector?

Answer: They can contain malicious prompts, oversized content, malformed PDFs, or sensitive data.

7. How do you validate file uploads?

Answer: Check extension, MIME type, size, content parser safety, file name sanitization, and scan if required.

8. Why is MD5 weak for security?

Answer: MD5 has collision weaknesses. It is acceptable for simple duplicate detection but not for security-sensitive integrity checks.

9. Should uploaded files be publicly served from /uploads?

Answer: Usually no. Private documents should be protected with auth and signed URLs or stored privately.

10. How would you secure Redis, Postgres, and Ollama?

Answer: Use private networks, authentication, TLS where possible, strong credentials, least privilege, backups, and monitoring.

Scenario Questions

1. User asks a question, but answer is wrong. How do you debug?

Answer: Check retrieved chunks, reranker output, prompt, tenant/category filters, source document quality, and LLM temperature.

2. Uploaded document is not searchable. What do you check?

Answer: Check Celery task status, document status, extraction logs, vector insert success, tenant metadata, and embedding dimension.

3. Chat response is very slow. What could be the cause?

Answer: Slow retrieval, reranking, local LLM generation, Redis latency, DB connection pool pressure, or too many concurrent workers.

4. RAG retrieves irrelevant chunks. How do you fix it?

Answer: Improve chunking, metadata filters, query rewriting, embedding model, top-k settings, reranking, and category routing.

5. Reranker improves accuracy but increases latency. What do you do?

Answer: Reduce candidate top-k, use a smaller reranker, cache results, or make reranking optional based on query type.

6. Redis is down. What features fail?

Answer: Celery broker/backend, chat history, semantic cache, and token blacklist checks can fail.

7. Pgvector table grows very large. How do you optimize?

Answer: Add indexes, tune HNSW, archive/delete old vectors, partition data, limit metadata, and monitor query plans.

8. LLM returns no usage metadata. What breaks?

Answer: Code that directly accesses input_tokens, output_tokens, or total_tokens can raise KeyError during DB logging.

9. Two tenants upload same file. Should it be allowed?

Answer: Yes, if tenant data is isolated. Duplicate checks should usually be per tenant, not global.

10. A user changes embedding model. What must be done?

Answer: Re-embed all documents and recreate compatible vector indexes/tables if the dimension changes.

Strong Project Explanation

Question: Explain your project.

Answer: This is a multi-tenant GenAI chatbot platform using FastAPI, WebSocket streaming, Celery, Redis, PostgreSQL with pgvector, local HuggingFace embeddings, a CrossEncoder reranker, and an Ollama-based LLM.

Tenant admins upload PDF documents. The API saves the file, checks duplicates using file hash, creates a DB record, and sends a Celery task for background processing. The worker extracts text, chunks it, generates embeddings, attaches metadata like tenant_id, category, page number, and stores vectors in pgvector.

For chat, the frontend connects through WebSocket with agent_id and token. The backend validates the agent and tenant access, builds an agent service, retrieves relevant chunks from pgvector, reranks them, passes context to the LLM, and streams tokens back to the client. It also stores conversation, messages, usage logs, and Redis chat history.